1. **Aim of the practical:** Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

2. **Objective:** To implement Kruskal's algorithm in C++ to find the Minimum Cost Spanning Tree of an undirected graph.

3. **Commands/Steps:**

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

vector<int> parent, ranks;

int find(int i) {
```

```
    if (parent[i] != i)
        parent[i] = find(parent[i]);
    return parent[i];
}

void unionSet(int x, int y) {
    int xroot = find(x);
    int yroot = find(y);

    if (ranks[xroot] < ranks[yroot]) {
        parent[xroot] = yroot;
    } else if (ranks[xroot] > ranks[yroot]) {
        parent[yroot] = xroot;
    } else {
        parent[yroot] = xroot;
        ranks[xroot]++;
    }
}

int main() {
    int V, E;
    cout << "Enter number of vertices and edges: ";
    cin >> V >> E;

    vector<Edge> edges(E);
    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < E; ++i) {
        cin >> edges[i].u >> edges[i].v >> edges[i].weight;
    }

    sort(edges.begin(), edges.end(), compare);

    parent.resize(V);
    ranks.resize(V, 0);

    for (int i = 0; i < V; ++i)
        parent[i] = i;

    int cost = 0;
```

```
    cout << "\nEdges in Minimum Spanning Tree:\n";
    for (auto edge : edges) {
        int setU = find(edge.u);
        int setV = find(edge.v);
        if (setU != setV) {
            cout << edge.u << " - " << edge.v << " : " << edge.weight << endl;
            cost += edge.weight;
            unionSet(setU, setV);
        }
    }

    cout << "\nTotal Minimum Cost: " << cost << endl;

    return 0;
}
```

4. Output:

```
PS C:\Users\AARYAMAN\Desktop\MCA_ALL SUbje
ktop\MCA_ALL SUbjectsNOtes\SEMESTER-2\DAA
in }
Enter number of vertices and edges: 4 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in Minimum Spanning Tree:
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10

Total Minimum Cost: 19
```

5. Learning Outcomes (what I have learned):

- Understand and apply the concept of Greedy algorithms in graph theory.
- Gain hands-on experience with Disjoint Set Union (Union-Find) data structure for cycle detection.
- Develop the ability to construct and sort edges efficiently to build a Minimum Spanning Tree.